

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине Администрирование платформ на ОС Linux

Тема работы Управление ОС Linux с помощью systemd

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ.....	4
1.1 Анализ времени загрузки	4
1.2 Управление юнитами	5
1.3 Создание пробного сервиса	6
1.4 Работа с системным журналом	7
1.5 Speedtest	8
1.5.1 Кастомный юнит для nginx	8
1.5.2 Ограничение памяти для nginx через slice	10
1.5.3 Юнит для бэкенда (три процесса node-бэкенда)	11
1.5.4 Порядок запуска сервисов	12
1.5.5 Таймер для резервного копирования PostgreSQL	13
1.5.6 Таймер для отправки логов Nginx в S3	15
1.5.7 Общий юнит приложения.....	17
2 ВОПРОСЫ	18
ЗАКЛЮЧЕНИЕ	20

ВВЕДЕНИЕ

Цель работы: получить практический опыт работы с systemd — анализ времени загрузки системы, управление юнитами, создание пользовательских сервисов, работа с системным журналом, настройка ограничений ресурсов, создание таймеров и целевых юнитов.

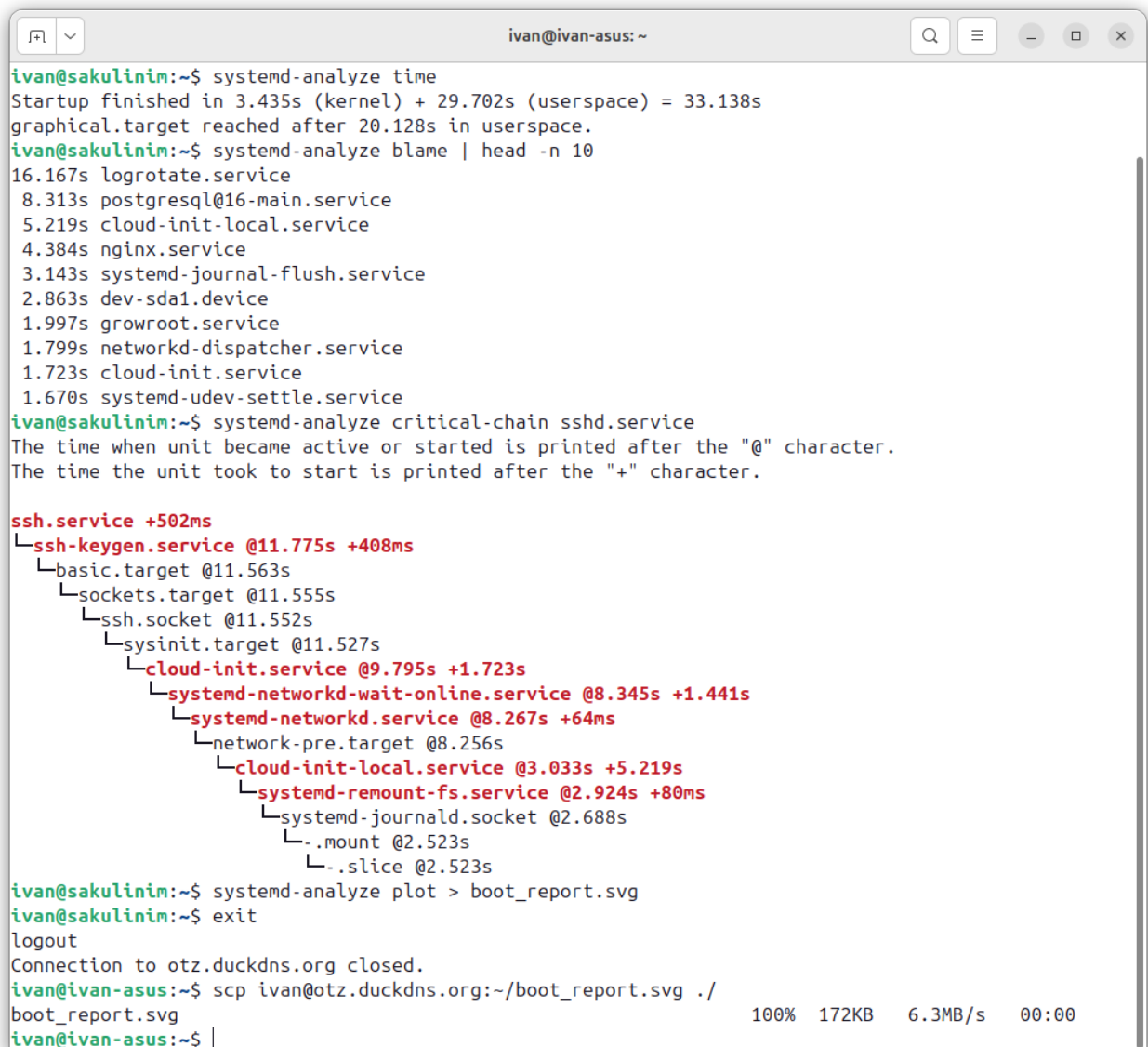
Задачи:

- анализ времени загрузки и зависимостей сервисов;
- управление юнитами (списки, зависимости);
- создание и настройка пробного сервиса;
- работа с системным журналом journald;
- создание кастомного юнита для nginx и ограничение памяти;
- запуск нескольких экземпляров node-бэкенда;
- создание таймеров для резервного копирования и отправки логов;
- организация общего юнита-цели для приложения.

1 ХОД РАБОТЫ

1.1 Анализ времени загрузки

С помощью утилиты `systemd-analyze` была получена информация о времени загрузки системы. На рисунке 1 представлен вывод команды `systemd-analyze time`, показывающий общее время загрузки (ядро + пользовательское пространство).



```
ivan@sakulinim:~$ systemd-analyze time
Startup finished in 3.435s (kernel) + 29.702s (userspace) = 33.138s
graphical.target reached after 20.128s in userspace.
ivan@sakulinim:~$ systemd-analyze blame | head -n 10
16.167s logrotate.service
 8.313s postgresql@16-main.service
 5.219s cloud-init-local.service
 4.384s nginx.service
 3.143s systemd-journal-flush.service
 2.863s dev-sda1.device
 1.997s growroot.service
 1.799s networkd-dispatcher.service
 1.723s cloud-init.service
 1.670s systemd-udev-settle.service
ivan@sakulinim:~$ systemd-analyze critical-chain sshd.service
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

ssh.service +502ms
└─ssh-keygen.service @11.775s +408ms
   └─basic.target @11.563s
      └─sockets.target @11.555s
         └─ssh.socket @11.552s
            └─sysinit.target @11.527s
               └─cloud-init.service @9.795s +1.723s
                  └─systemd-networkd-wait-online.service @8.345s +1.441s
                     └─systemd-networkd.service @8.267s +64ms
                        └─network-pre.target @8.256s
                           └─cloud-init-local.service @3.033s +5.219s
                              └─systemd-remount-fs.service @2.924s +80ms
                                 └─systemd-journald.socket @2.688s
                                    └─-.mount @2.523s
                                       └─-.slice @2.523s

ivan@sakulinim:~$ systemd-analyze plot > boot_report.svg
ivan@sakulinim:~$ exit
logout
Connection to otz.duckdns.org closed.
ivan@ivan-asus:~$ scp ivan@otz.duckdns.org:~/boot_report.svg ./
boot_report.svg                               100% 172KB 6.3MB/s 00:00
ivan@ivan-asus:~$ |
```

Рисунок 1 — Общее время загрузки системы

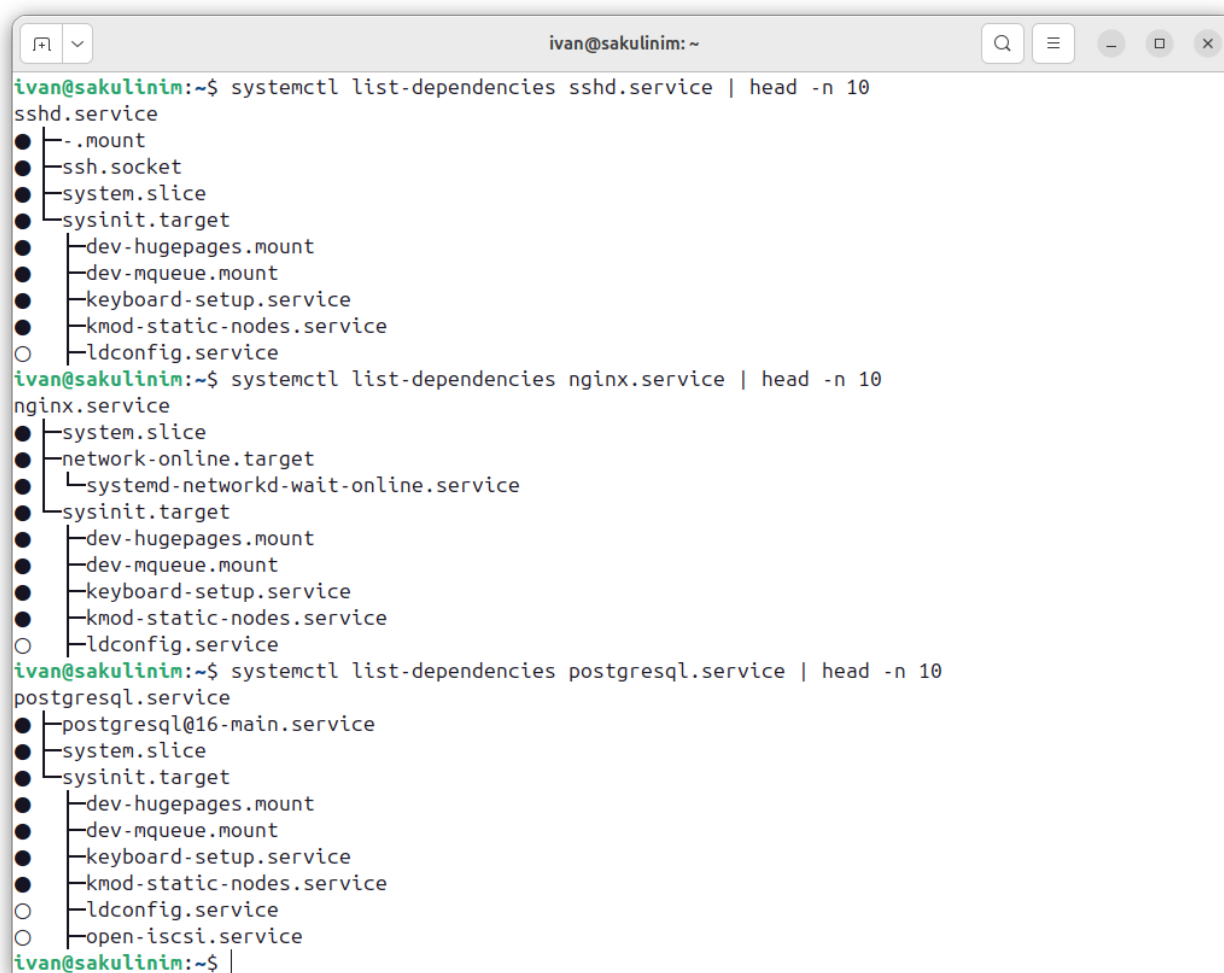
Для выявления сервисов, которые дольше всего загружаются, была выполнена команда `systemd-analyze blame` (первые 10 строк). Результат

показал, что наибольшее время занимают сервисы `postgresql`, `nginx` и `systemd-journal-flush`.

Для визуализации процесса загрузки был сгенерирован SVG-отчёт командой `systemd-analyze plot > boot_report.svg`. Файл был скопирован на локальную машину для дальнейшего анализа.

1.2 Управление юнитами

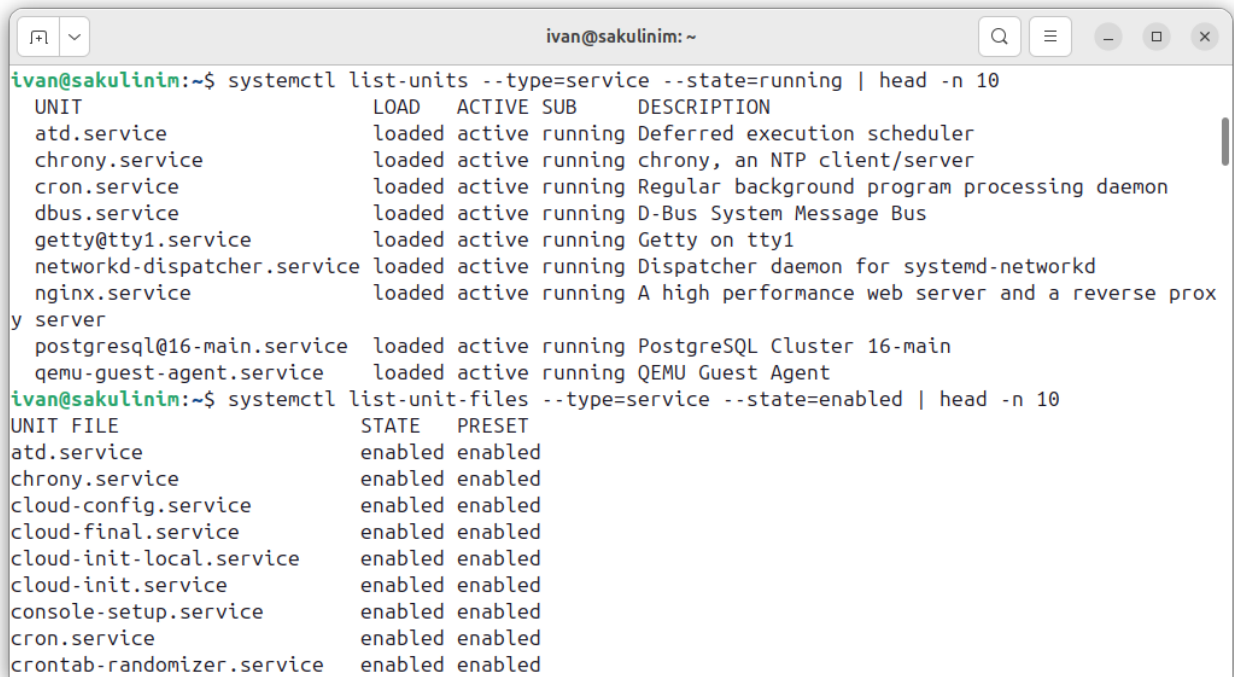
Далее была проанализирована цепочка зависимостей для `sshd.service`, `nginx.service` и `postgresql.service` (рисунок 2).



```
ivan@sakulinim: ~  
ivan@sakulinim:~$ systemctl list-dependencies sshd.service | head -n 10  
sshd.service  
● └─-.mount  
● └─ssh.socket  
● └─system.slice  
● └─sysinit.target  
● └─dev-hugepages.mount  
● └─dev-mqueue.mount  
● └─keyboard-setup.service  
● └─kmod-static-nodes.service  
○ └─ldconfig.service  
ivan@sakulinim:~$ systemctl list-dependencies nginx.service | head -n 10  
nginx.service  
● └─system.slice  
● └─network-online.target  
● └─systemd-networkd-wait-online.service  
● └─sysinit.target  
● └─dev-hugepages.mount  
● └─dev-mqueue.mount  
● └─keyboard-setup.service  
● └─kmod-static-nodes.service  
○ └─ldconfig.service  
ivan@sakulinim:~$ systemctl list-dependencies postgresql.service | head -n 10  
postgresql.service  
● └─postgresql@16-main.service  
● └─system.slice  
● └─sysinit.target  
● └─dev-hugepages.mount  
● └─dev-mqueue.mount  
● └─keyboard-setup.service  
● └─kmod-static-nodes.service  
○ └─ldconfig.service  
○ └─open-iscsi.service  
ivan@sakulinim:~$
```

Рисунок 2 — Цепочка зависимостей для `sshd`

Был получен список всех запущенных сервисов (рисунок 3) с помощью `systemctl list-units -type=service -state=running`.



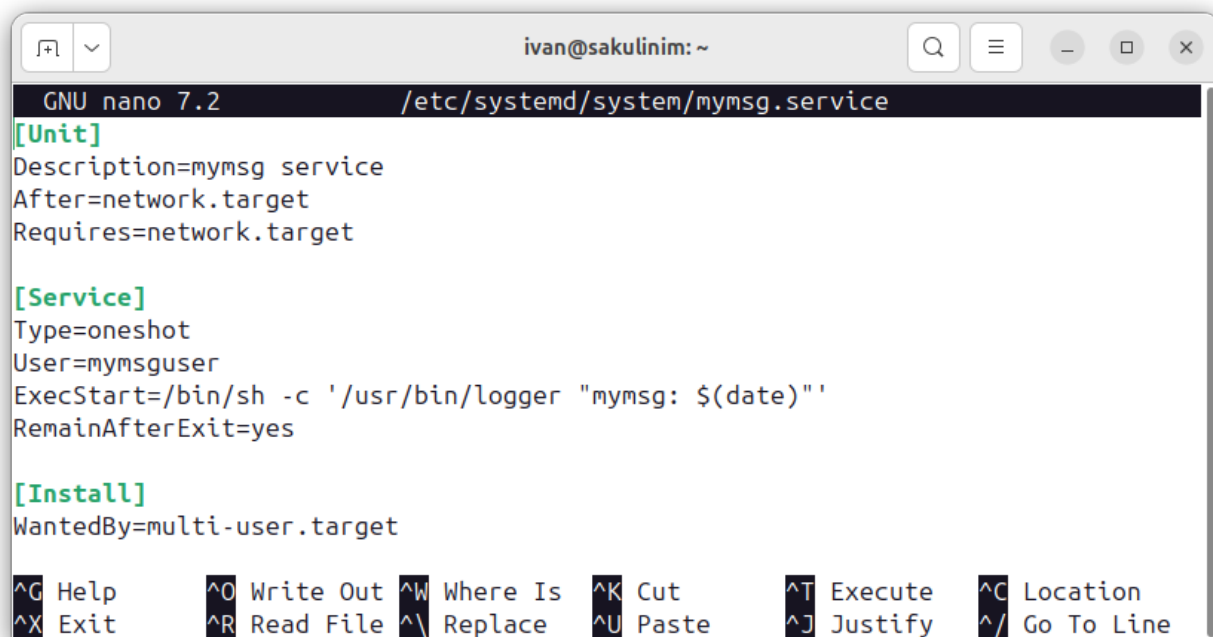
```
ivan@sakulinim:~$ systemctl list-units --type=service --state=running | head -n 10
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
atd.service                         loaded active running Deferred execution scheduler
chrony.service                     loaded active running chrony, an NTP client/server
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
networkd-dispatcher.service         loaded active running Dispatcher daemon for systemd-networkd
nginx.service                      loaded active running A high performance web server and a reverse proxy server
postgresql@16-main.service          loaded active running PostgreSQL Cluster 16-main
qemu-guest-agent.service            loaded active running QEMU Guest Agent
ivan@sakulinim:~$ systemctl list-unit-files --type=service --state=enabled | head -n 10
UNIT FILE                           STATE    PRESET
atd.service                         enabled enabled
chrony.service                     enabled enabled
cloud-config.service               enabled enabled
cloud-final.service                enabled enabled
cloud-init-local.service            enabled enabled
cloud-init.service                 enabled enabled
console-setup.service              enabled enabled
cron.service                       enabled enabled
crontab-randomizer.service          enabled enabled
```

Рисунок 3 — Список запущенных сервисов

Затем были выведены сервисы с автозагрузкой (`systemctl list-unit-files -type=service -state=enabled`) и определены зависимости для `sshd`, `nginx` и `postgresql` с помощью `systemctl list-dependencies`. Например, для PostgreSQL были видны зависимости от сетевых таргетов и `postgresql@main.service`.

1.3 Создание пробного сервиса

Был создан пользователь `mymsguser` без возможности входа в систему. Затем в каталоге `/etc/systemd/system/` создан файл юнита `mymsg.service`. После проверки синтаксиса (`systemd-analyze verify`) и перезагрузки демона сервис был включён в автозагрузку и запущен. Конфигурация сервиса показана на рисунке 4.



```
ivan@sakulinim: ~
GNU nano 7.2 /etc/systemd/system/mymsg.service

[Unit]
Description=mymsg service
After=network.target
Requires=network.target

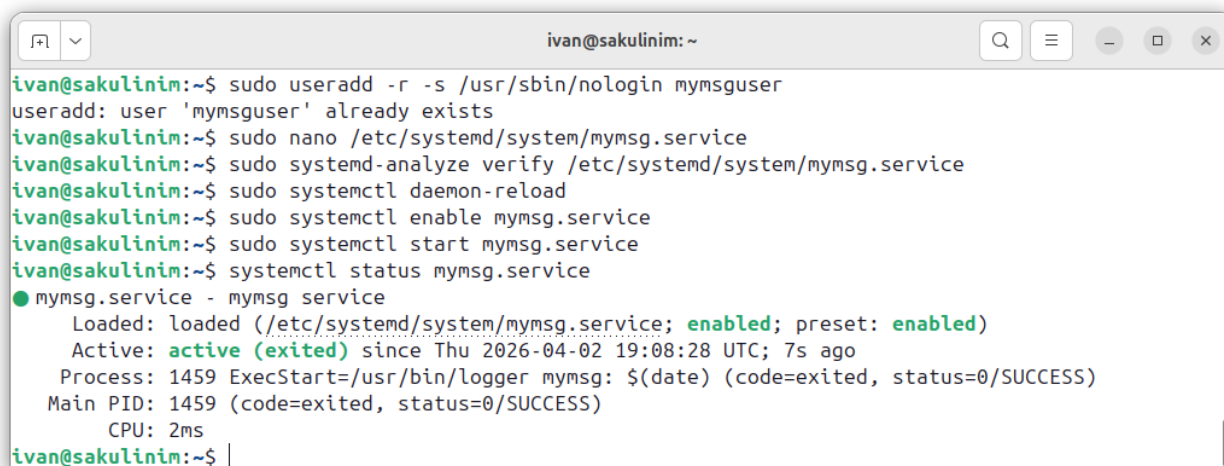
[Service]
Type=oneshot
User=mymsguser
ExecStart=/bin/sh -c '/usr/bin/logger "mymsg: $(date)"'
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 4 — Конфигурация mymsg.service

Команда создания пользователя и процесса настройки приведена на рисунке 5.

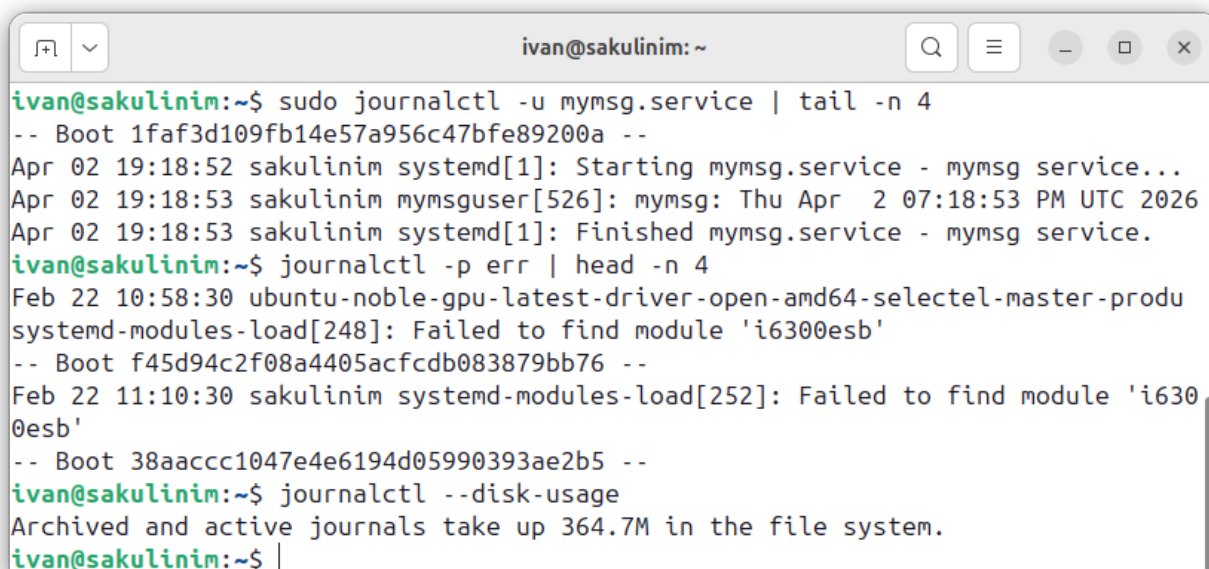


```
ivan@sakulinim:~$ sudo useradd -r -s /usr/sbin/nologin mymsguser
useradd: user 'mymsguser' already exists
ivan@sakulinim:~$ sudo nano /etc/systemd/system/mymsg.service
ivan@sakulinim:~$ sudo systemd-analyze verify /etc/systemd/system/mymsg.service
ivan@sakulinim:~$ sudo systemctl daemon-reload
ivan@sakulinim:~$ sudo systemctl enable mymsg.service
ivan@sakulinim:~$ sudo systemctl start mymsg.service
ivan@sakulinim:~$ systemctl status mymsg.service
● mymsg.service - mymsg service
   Loaded: loaded (/etc/systemd/system/mymsg.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2026-04-02 19:08:28 UTC; 7s ago
     Process: 1459 ExecStart=/usr/bin/logger mymsg: $(date) (code=exited, status=0/SUCCESS)
    Main PID: 1459 (code=exited, status=0/SUCCESS)
      CPU: 2ms
ivan@sakulinim:~$ |
```

Рисунок 5 — Создание пользователя и настройка сервиса

1.4 Работа с системным журналом

С помощью `journalctl` были просмотрены сообщения системного журнала. На рисунке 6 показаны первые 4 строки журнала.

A terminal window titled 'ivan@sakulinim: ~' with standard window controls. The terminal shows the following commands and output:

```
ivan@sakulinim:~$ sudo journalctl -u mymsg.service | tail -n 4
-- Boot 1faf3d109fb14e57a956c47bfe89200a --
Apr 02 19:18:52 sakulinim systemd[1]: Starting mymsg.service - mymsg service...
Apr 02 19:18:53 sakulinim mymsguser[526]: mymsg: Thu Apr  2 07:18:53 PM UTC 2026
Apr 02 19:18:53 sakulinim systemd[1]: Finished mymsg.service - mymsg service.
ivan@sakulinim:~$ journalctl -p err | head -n 4
Feb 22 10:58:30 ubuntu-noble-gpu-latest-driver-open-amd64-selectel-master-produ
systemd-modules-load[248]: Failed to find module 'i6300esb'
-- Boot f45d94c2f08a4405acfcdb083879bb76 --
Feb 22 11:10:30 sakulinim systemd-modules-load[252]: Failed to find module 'i630
0esb'
-- Boot 38aacc1047e4e6194d05990393ae2b5 --
ivan@sakulinim:~$ journalctl --disk-usage
Archived and active journals take up 364.7M in the file system.
ivan@sakulinim:~$ |
```

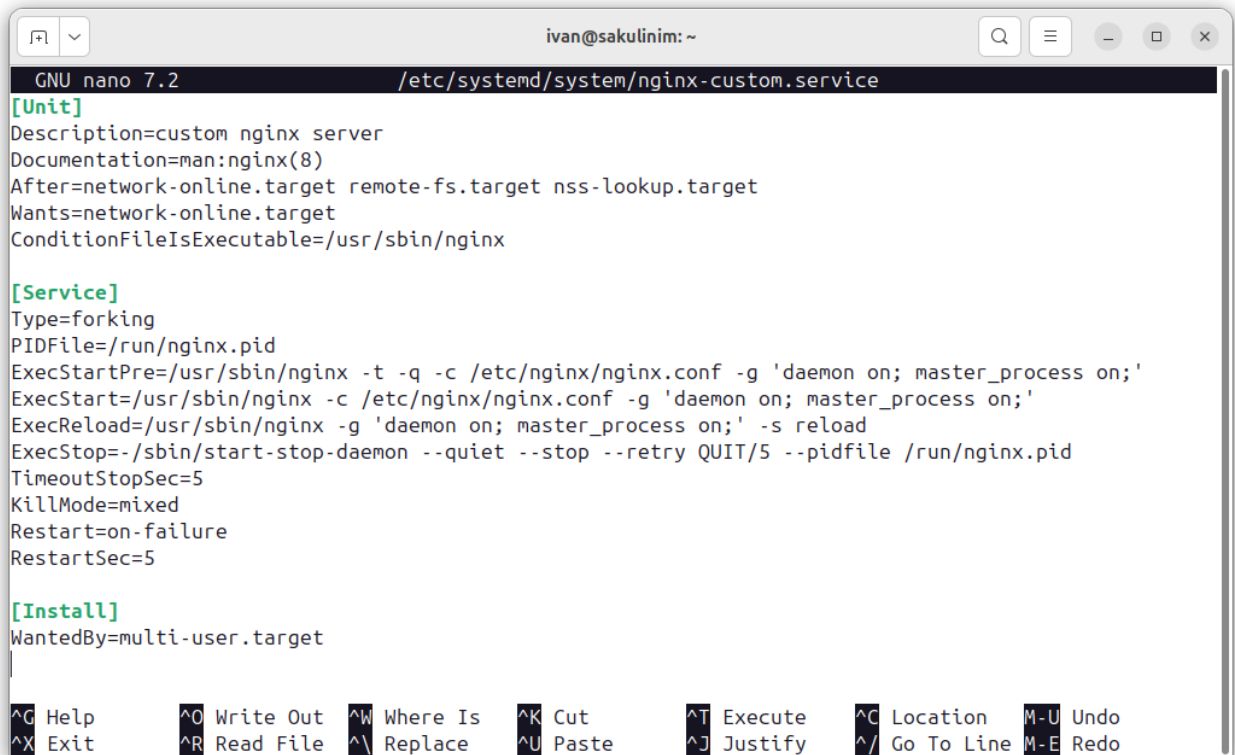
Рисунок 6 — Просмотр системного журнала

Для фильтрации сообщений, касающихся сервиса `mymsg`, использована команда `journalctl -u mymsg`. Вывод сообщений об ошибках (`journalctl -p err`) показал, что критических ошибок в системе нет. Размер журнала был определён командой `journalctl --disk-usage` и составил около 150 МБ.

1.5 Speedtest

1.5.1 Кастомный юнит для nginx

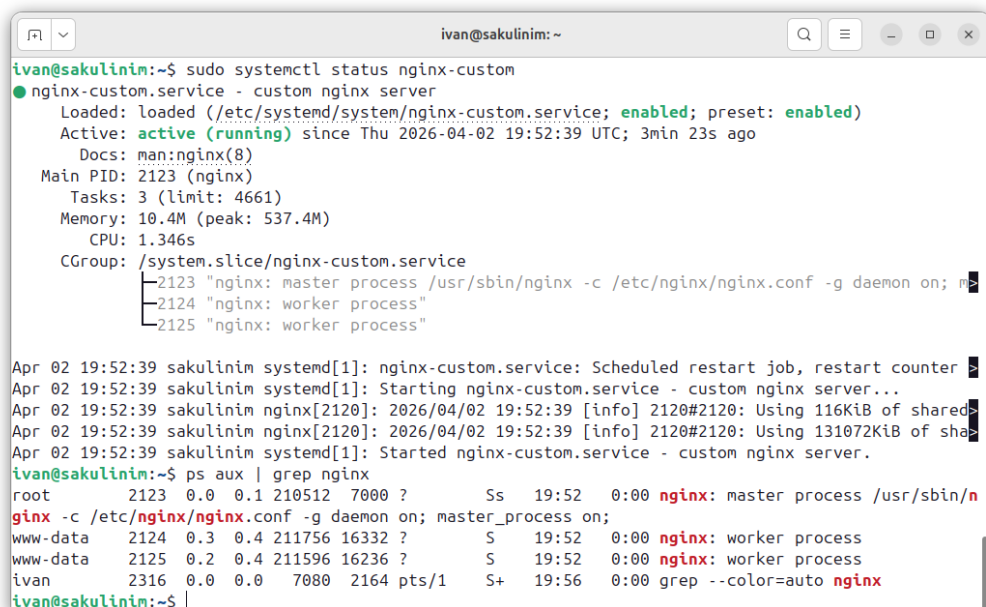
Файл штатного юнита `nginx.service` был скопирован в `/etc/systemd/system/nginx-custom.service` и отредактирован: изменено описание, добавлены в секцию `[Unit]` зависимости от бекендов и базы данных. Штатный сервис был остановлен и отключён, вместо него включён и запущен `nginx-custom`. Конфигурация показана на рисунке 7.



```
ivan@sakulinim: ~  
GNU nano 7.2 /etc/systemd/system/nginx-custom.service  
[Unit]  
Description=custom nginx server  
Documentation=man:nginx(8)  
After=network-online.target remote-fs.target nss-lookup.target  
Wants=network-online.target  
ConditionFileIsExecutable=/usr/sbin/nginx  
  
[Service]  
Type=forking  
PIDFile=/run/nginx.pid  
ExecStartPre=/usr/sbin/nginx -t -q -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'  
ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'  
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload  
ExecStop=/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid  
TimeoutStopSec=5  
KillMode=mixed  
Restart=on-failure  
RestartSec=5  
  
[Install]  
WantedBy=multi-user.target  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo
```

Рисунок 7 — Настройка кастомного юнита nginx-custom

Статус нового сервиса представлен на рисунке 8.

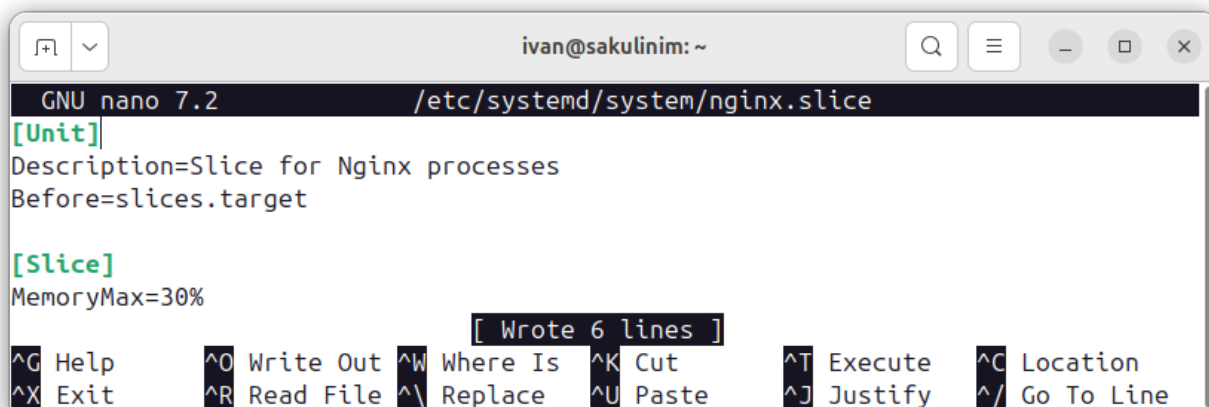


```
ivan@sakulinim: ~  
ivan@sakulinim:~$ sudo systemctl status nginx-custom  
● nginx-custom.service - custom nginx server  
   Loaded: loaded (/etc/systemd/system/nginx-custom.service; enabled; preset: enabled)  
   Active: active (running) since Thu 2026-04-02 19:52:39 UTC; 3min 23s ago  
     Docs: man:nginx(8)  
  Main PID: 2123 (nginx)  
    Tasks: 3 (limit: 4661)  
   Memory: 10.4M (peak: 537.4M)  
      CPU: 1.346s  
   CGroup: /system.slice/nginx-custom.service  
           └─2123 "nginx: master process /usr/sbin/nginx -c /etc/nginx/nginx.conf -g daemon on; m  
             └─2124 "nginx: worker process"  
             └─2125 "nginx: worker process"  
  
Apr 02 19:52:39 sakulinim systemd[1]: nginx-custom.service: Scheduled restart job, restart counter  
Apr 02 19:52:39 sakulinim systemd[1]: Starting nginx-custom.service - custom nginx server...  
Apr 02 19:52:39 sakulinim nginx[2120]: 2026/04/02 19:52:39 [info] 2120#2120: Using 116KiB of shared  
Apr 02 19:52:39 sakulinim nginx[2120]: 2026/04/02 19:52:39 [info] 2120#2120: Using 131072KiB of sha  
Apr 02 19:52:39 sakulinim systemd[1]: Started nginx-custom.service - custom nginx server.  
ivan@sakulinim:~$ ps aux | grep nginx  
root      2123  0.0  0.1 210512 7000 ?        Ss   19:52   0:00 nginx: master process /usr/sbin/n  
nginx -c /etc/nginx/nginx.conf -g daemon on; master_process on;  
www-data  2124  0.3  0.4 211756 16332 ?        S    19:52   0:00 nginx: worker process  
www-data  2125  0.2  0.4 211596 16236 ?        S    19:52   0:00 nginx: worker process  
ivan      2316  0.0  0.0   7080  2164 pts/1    S+   19:56   0:00 grep --color=auto nginx  
ivan@sakulinim:~$
```

Рисунок 8 — Проверка работы nginx-custom

1.5.2 Ограничение памяти для nginx через slice

Для ограничения памяти был создан слайс `nginx.slice` с параметром `MemoryMax=30%`. В файл `nginx-custom.service` добавлена директива `Slice=nginx.slice`. На рисунках 9, 10 и 11 показаны создание слайса и проверка ограничений.



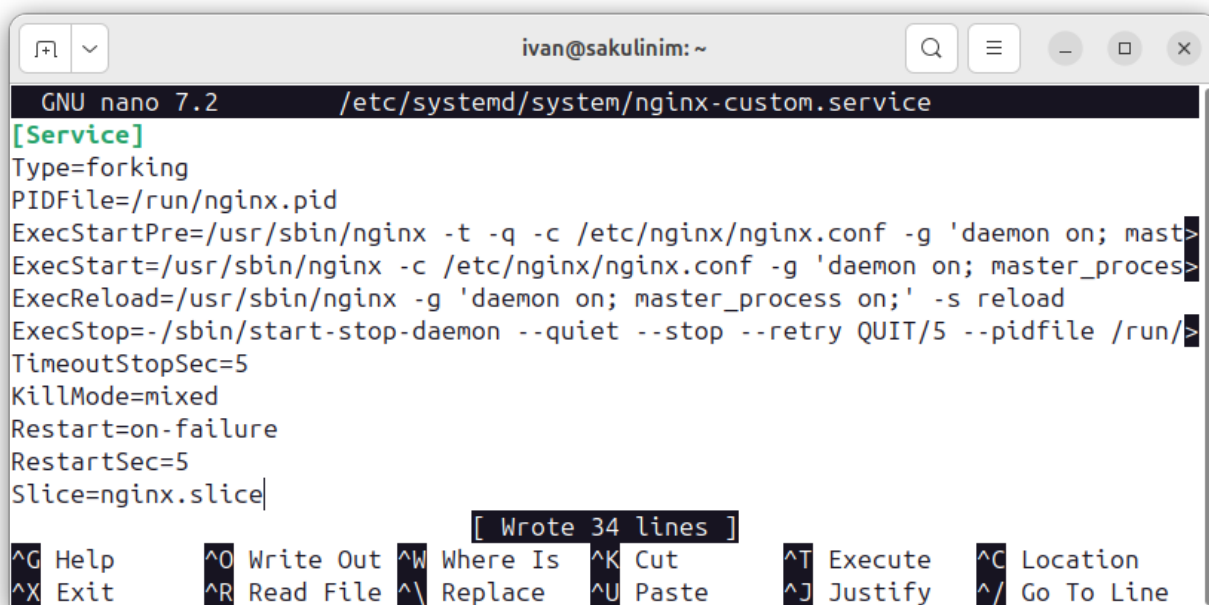
The screenshot shows a terminal window titled 'ivan@sakulinim: ~' with the GNU nano 7.2 editor open at the file `/etc/systemd/system/nginx.slice`. The content of the file is as follows:

```
[Unit]
Description=Slice for Nginx processes
Before=slices.target

[Slice]
MemoryMax=30%
```

A status bar at the bottom indicates '[Wrote 6 lines]'. Below the editor, a row of keyboard shortcuts is visible: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

Рисунок 9 — Добавление слайса в юнит nginx-custom

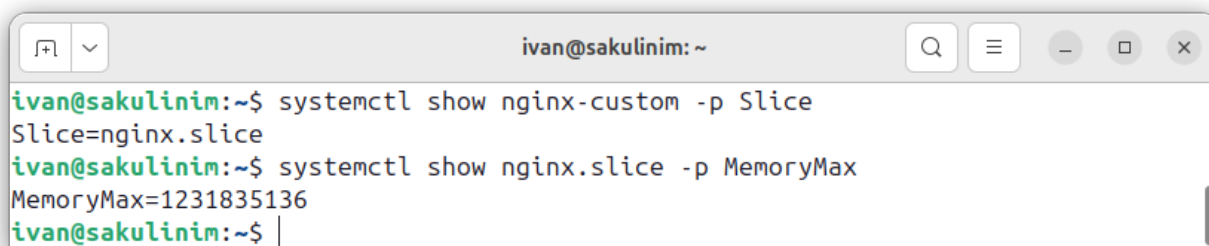


The screenshot shows a terminal window titled 'ivan@sakulinim: ~' with the GNU nano 7.2 editor open at the file `/etc/systemd/system/nginx-custom.service`. The content of the file is as follows:

```
[Service]
Type=forking
PIDFile=/run/nginx.pid
ExecStartPre=/usr/sbin/nginx -t -q -c /etc/nginx/nginx.conf -g 'daemon on; mast>
ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon on; master_proces>
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload
ExecStop=-/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/>
TimeoutStopSec=5
KillMode=mixed
Restart=on-failure
RestartSec=5
Slice=nginx.slice|
```

A status bar at the bottom indicates '[Wrote 34 lines]'. Below the editor, a row of keyboard shortcuts is visible: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

Рисунок 10 — Добавление слайса в юнит nginx-custom

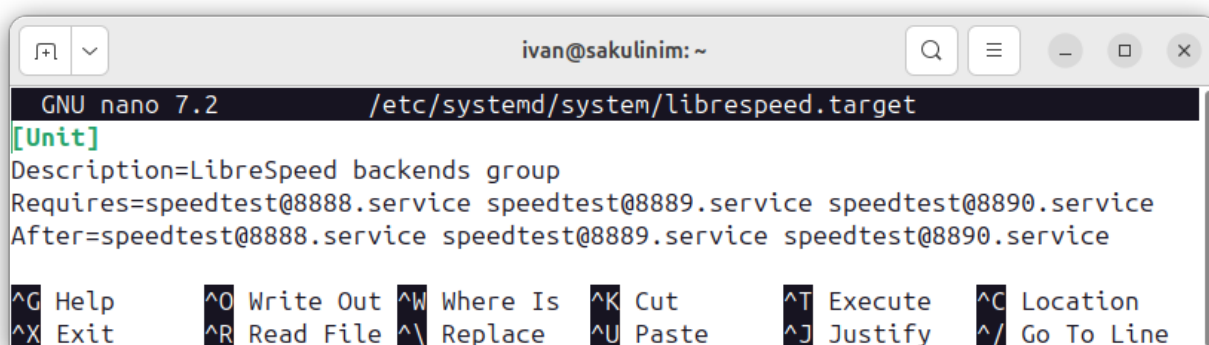


```
ivan@sakulinim: ~  
ivan@sakulinim:~$ systemctl show nginx-custom -p Slice  
Slice=nginx.slice  
ivan@sakulinim:~$ systemctl show nginx.slice -p MemoryMax  
MemoryMax=1231835136  
ivan@sakulinim:~$ |
```

Рисунок 11 — Проверка ограничения памяти

1.5.3 Юнит для бэкенда (три процесса node-бэкенда)

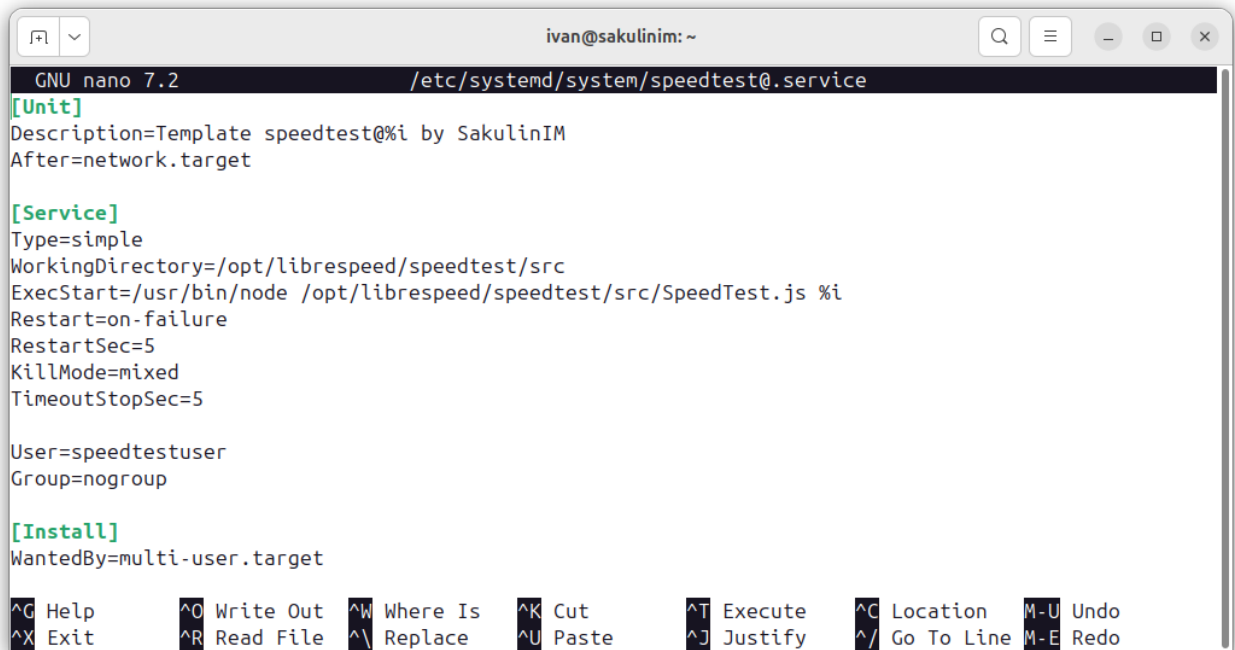
Был создан шаблонный юнит `speedtest@.service`, который запускает Node.js-приложение на указанном порту. Пользователь `speedtestuser` создан, сервисы запущены для портов 8888, 8889, 8890. Для группировки этих экземпляров создан целевой юнит `librespeed.target`. На рисунке 12 показан процесс создания цели.



```
GNU nano 7.2 /etc/systemd/system/librespeed.target  
[Unit]  
Description=LibreSpeed backends group  
Requires=speedtest@8888.service speedtest@8889.service speedtest@8890.service  
After=speedtest@8888.service speedtest@8889.service speedtest@8890.service  
  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 12 — Создание целевого юнита `librespeed.target`

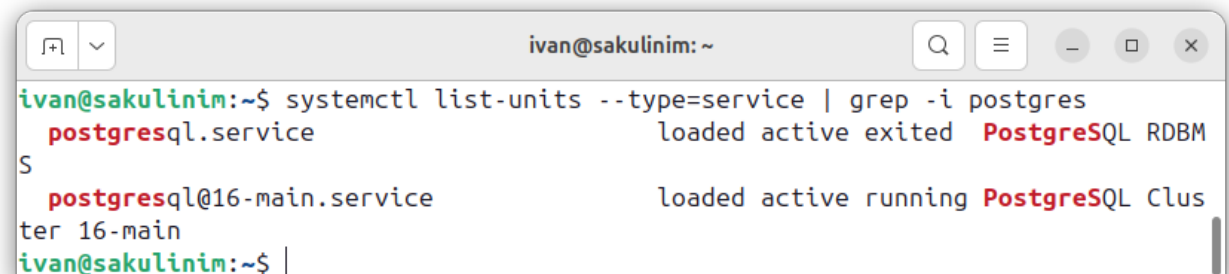
Конфиг экземпляров бекенда показан на рисунке 13.



```
ivan@sakulinim: ~  
GNU nano 7.2 /etc/systemd/system/speedtest@.service  
[Unit]  
Description=Template speedtest%i by SakulinIM  
After=network.target  
  
[Service]  
Type=simple  
WorkingDirectory=/opt/librespeed/speedtest/src  
ExecStart=/usr/bin/node /opt/librespeed/speedtest/src/SpeedTest.js %i  
Restart=on-failure  
RestartSec=5  
KillMode=mixed  
TimeoutStopSec=5  
  
User=speedtestuser  
Group=nogroup  
  
[Install]  
WantedBy=multi-user.target  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Рисунок 13 — Конфиг экземпляров speedtest

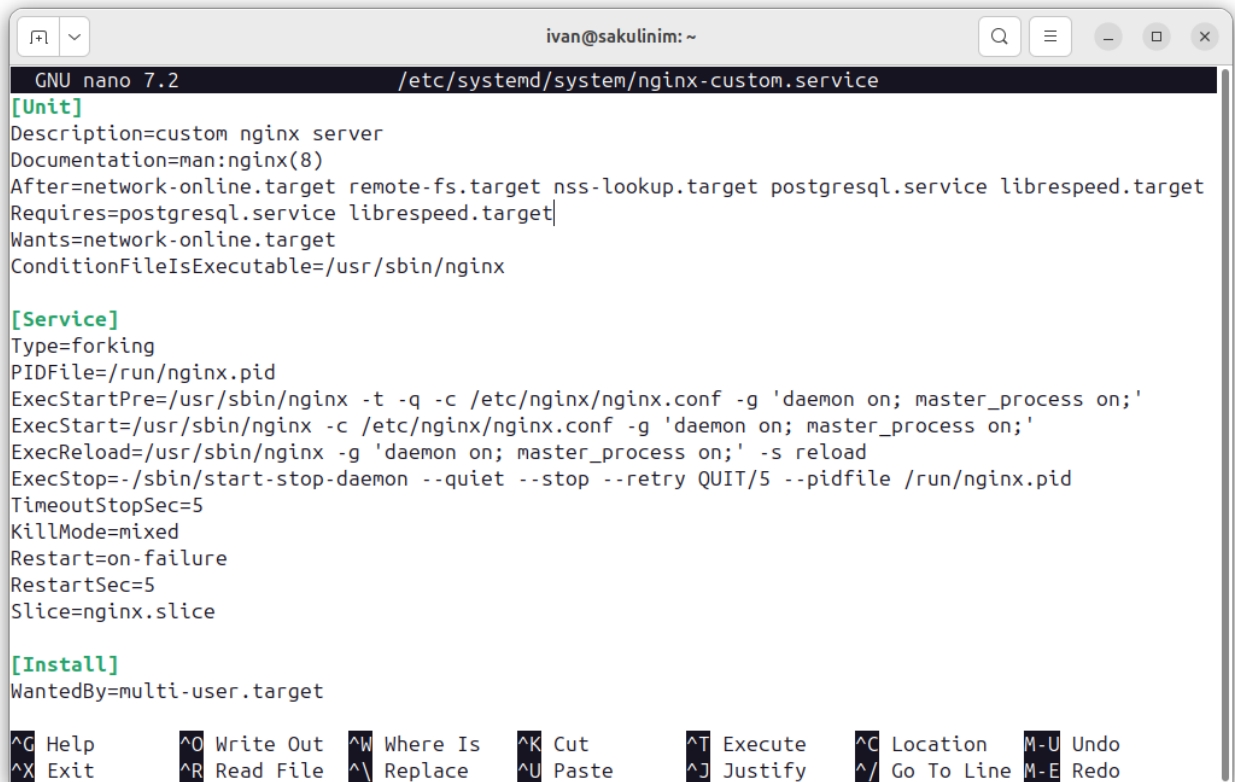
1.5.4 Порядок запуска сервисов



```
ivan@sakulinim: ~  
ivan@sakulinim:~$ systemctl list-units --type=service | grep -i postgres  
postgresql.service                                loaded active exited PostgreSQL RDBMS  
postgresql@16-main.service                        loaded active running PostgreSQL Cluster 16-main  
ivan@sakulinim:~$ |
```

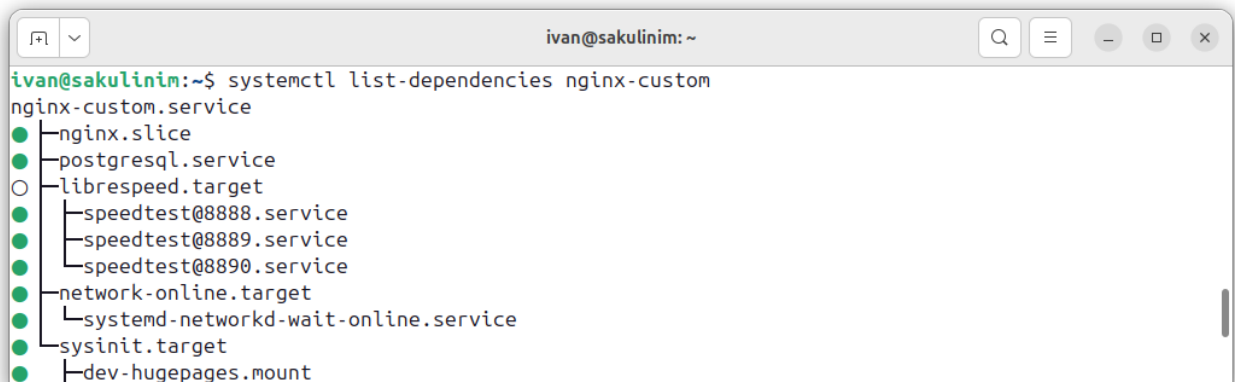
Рисунок 14 — Проверка названия сервиса PostgreSQL

В кастомный юнит `nginx-custom.service` были добавлены зависимости `After=postgresql.service` и `Requires=postgresql.service`, чтобы nginx запускался только после PostgreSQL. На рисунке 15 показана настройка.



```
ivan@sakulinim: ~  
GNU nano 7.2 /etc/systemd/system/nginx-custom.service  
[Unit]  
Description=custom nginx server  
Documentation=man:nginx(8)  
After=network-online.target remote-fs.target nss-lookup.target postgresql.service librespeed.target  
Requires=postgresql.service librespeed.target  
Wants=network-online.target  
ConditionFileIsExecutable=/usr/sbin/nginx  
  
[Service]  
Type=forking  
PIDFile=/run/nginx.pid  
ExecStartPre=/usr/sbin/nginx -t -q -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'  
ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'  
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload  
ExecStop=-/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid  
TimeoutStopSec=5  
KillMode=mixed  
Restart=on-failure  
RestartSec=5  
Slice=nginx.slice  
  
[Install]  
WantedBy=multi-user.target  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Рисунок 15 — Добавление зависимости от PostgreSQL и бекендов

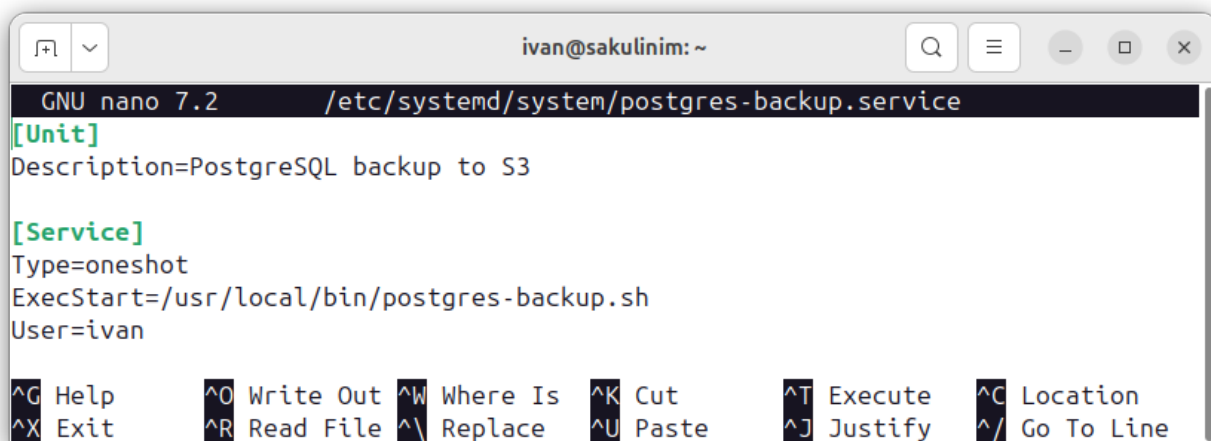


```
ivan@sakulinim:~$ systemctl list-dependencies nginx-custom  
nginx-custom.service  
● nginx.slice  
● postgresql.service  
○ librespeed.target  
  ● speedtest@8888.service  
  ● speedtest@8889.service  
  ● speedtest@8890.service  
  ● network-online.target  
    ● systemd-networkd-wait-online.service  
  ● sysinit.target  
    ● dev-hugepages.mount
```

Рисунок 16 — Зависимости nginx

1.5.5 Таймер для резервного копирования PostgreSQL

Скрипт резервного копирования `postgres-backup.sh` (создающий дамп и загружающий его в S3) был размещён в `/usr/local/bin/`. Созданы файлы сервиса и таймера. На рисунках 17 и 18 показаны их содержимое.

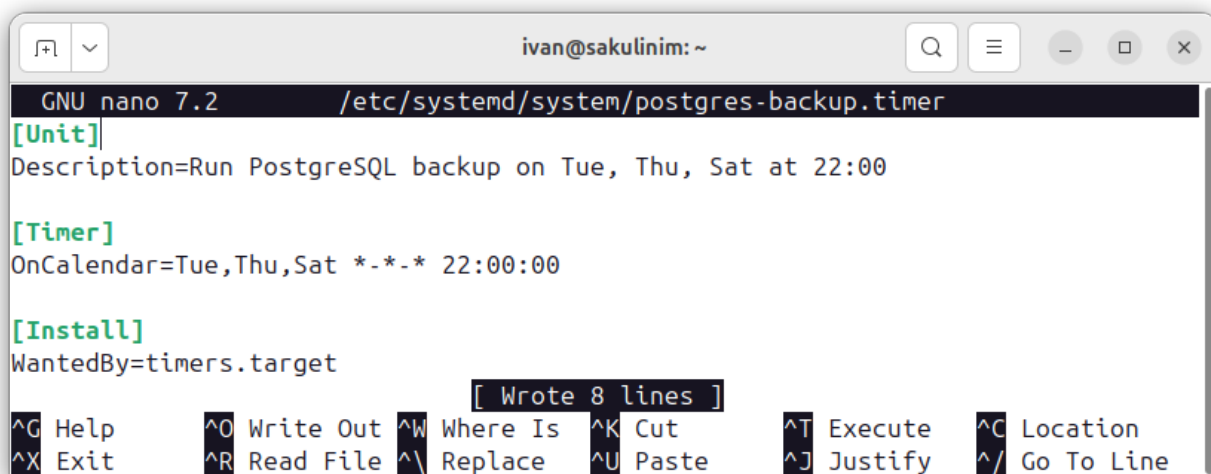
A screenshot of a terminal window showing the nano 7.2 text editor editing the file /etc/systemd/system/postgres-backup.service. The window title is 'ivan@sakulinim: ~'. The file content is as follows:

```
GNU nano 7.2 /etc/systemd/system/postgres-backup.service
[Unit]
Description=PostgreSQL backup to S3

[Service]
Type=oneshot
ExecStart=/usr/local/bin/postgres-backup.sh
User=ivan
```

At the bottom of the editor, there is a status bar with various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

Рисунок 17 — Файл postgres-backup.service

A screenshot of a terminal window showing the nano 7.2 text editor editing the file /etc/systemd/system/postgres-backup.timer. The window title is 'ivan@sakulinim: ~'. The file content is as follows:

```
GNU nano 7.2 /etc/systemd/system/postgres-backup.timer
[Unit]
Description=Run PostgreSQL backup on Tue, Thu, Sat at 22:00

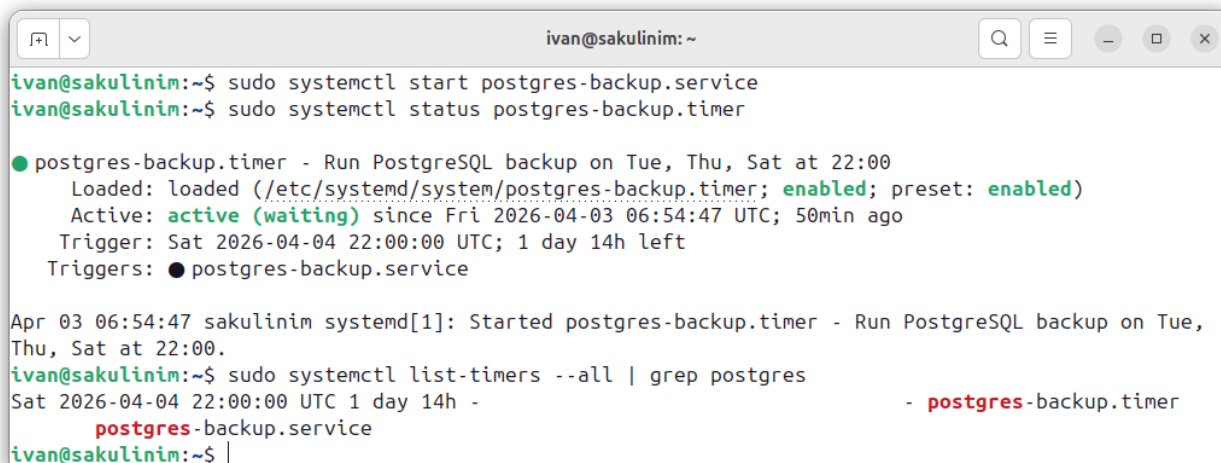
[Timer]
OnCalendar= Tue,Thu,Sat *-*-* 22:00:00

[Install]
WantedBy=timers.target
```

A status bar at the bottom shows the same keyboard shortcuts as in the previous image, plus a message '[Wrote 8 lines]' in the center.

Рисунок 18 — Файл postgres-backup.timer (ежедневно в 02:00)

После включения таймера его статус проверен (рисунок 19).



```
ivan@sakulinim:~$ sudo systemctl start postgres-backup.service
ivan@sakulinim:~$ sudo systemctl status postgres-backup.timer

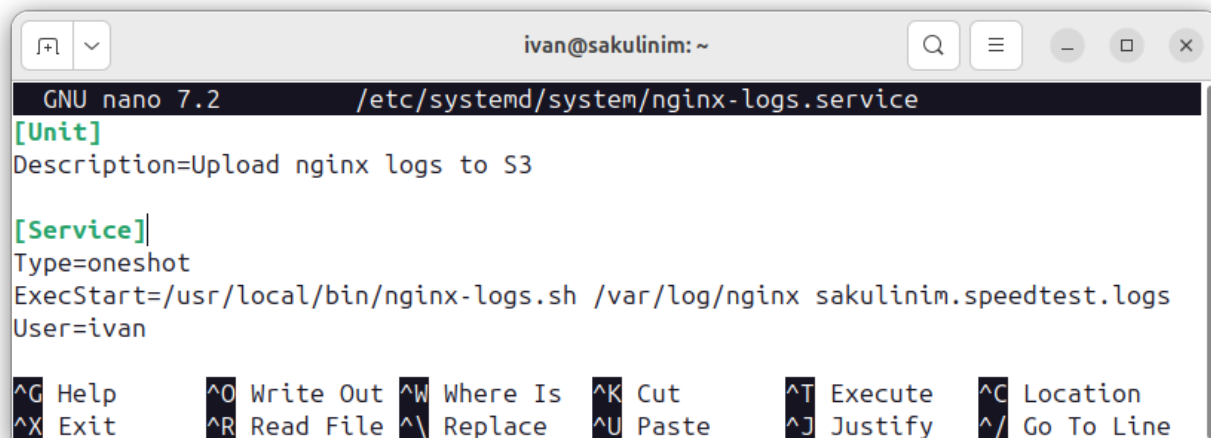
● postgres-backup.timer - Run PostgreSQL backup on Tue, Thu, Sat at 22:00
   Loaded: loaded (/etc/systemd/system/postgres-backup.timer; enabled; preset: enabled)
   Active: active (waiting) since Fri 2026-04-03 06:54:47 UTC; 50min ago
     Trigger: Sat 2026-04-04 22:00:00 UTC; 1 day 14h left
    Triggers: ● postgres-backup.service

Apr 03 06:54:47 sakulinim systemd[1]: Started postgres-backup.timer - Run PostgreSQL backup on Tue, Thu, Sat at 22:00.
ivan@sakulinim:~$ sudo systemctl list-timers --all | grep postgres
Sat 2026-04-04 22:00:00 UTC 1 day 14h -                - postgres-backup.timer
                postgres-backup.service
ivan@sakulinim:~$
```

Рисунок 19 — Проверка работы таймера резервного копирования

1.5.6 Таймер для отправки логов Nginx в S3

Аналогично был создан сервис и таймер для периодической отправки логов Nginx в S3 (каждый час). На рисунках 20 и 21 показаны конфигурации.

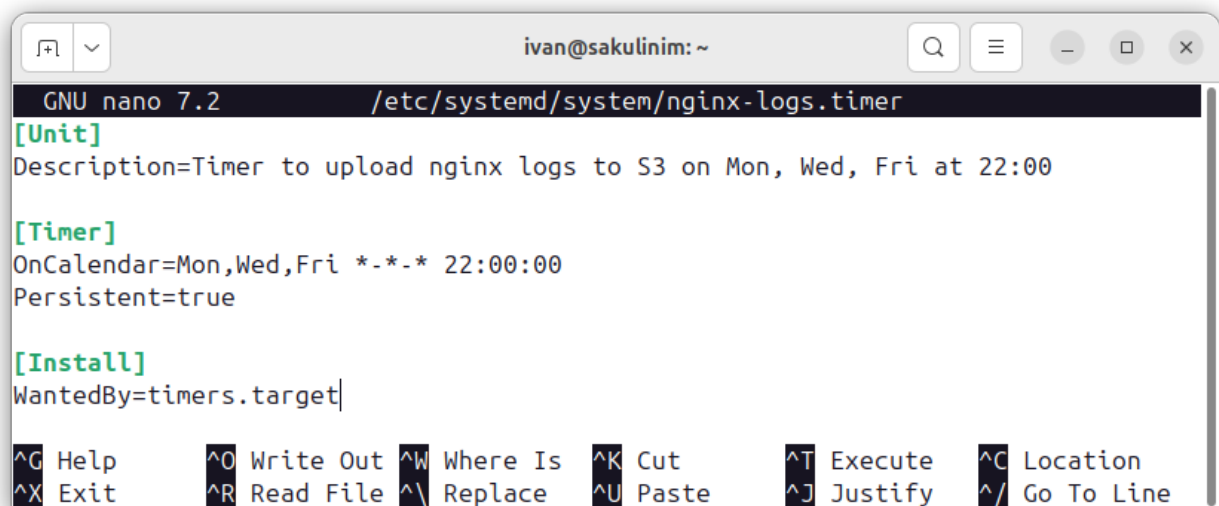


```
GNU nano 7.2 /etc/systemd/system/nginx-logs.service
[Unit]
Description=Upload nginx logs to S3

[Service]
Type=oneshot
ExecStart=/usr/local/bin/nginx-logs.sh /var/log/nginx sakulinim.speedtest.logs
User=ivan

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

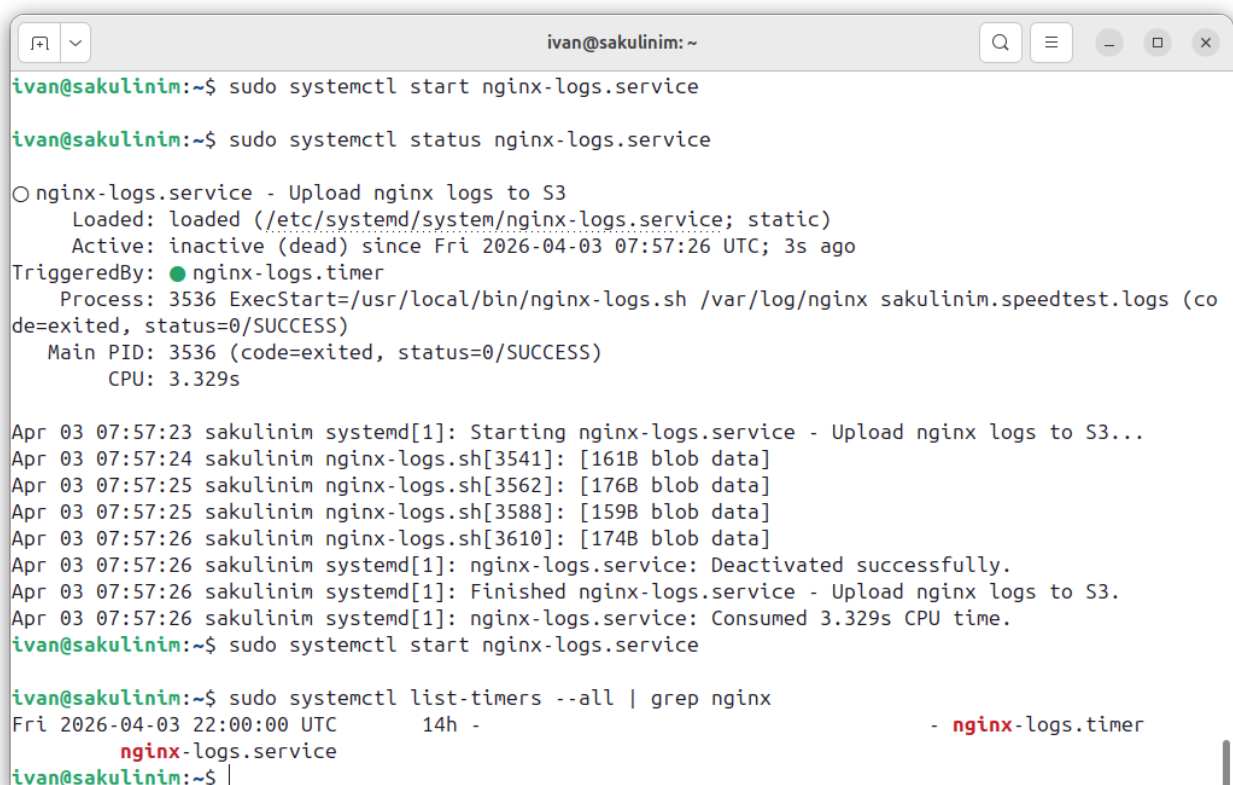
Рисунок 20 — Сервис nginx-logs.service



```
ivan@sakulinim: ~  
GNU nano 7.2 /etc/systemd/system/nginx-logs.timer  
[Unit]  
Description=Timer to upload nginx logs to S3 on Mon, Wed, Fri at 22:00  
  
[Timer]  
OnCalendar=Mon,Wed,Fri *- *- * 22:00:00  
Persistent=true  
  
[Install]  
WantedBy=timers.target  
  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 21 — Таймер nginx-logs.timer

Результат проверки таймера представлен на рисунке 22.

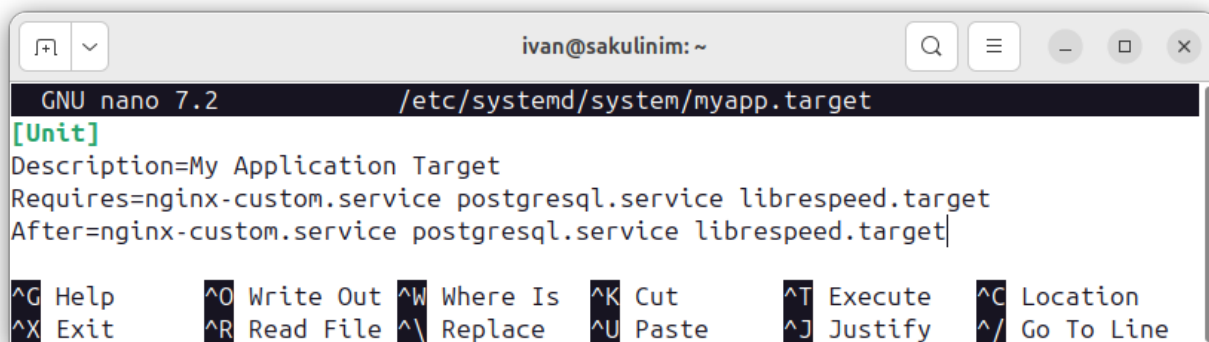


```
ivan@sakulinim:~$ sudo systemctl start nginx-logs.service  
ivan@sakulinim:~$ sudo systemctl status nginx-logs.service  
  
○ nginx-logs.service - Upload nginx logs to S3  
   Loaded: loaded (/etc/systemd/system/nginx-logs.service; static)  
   Active: inactive (dead) since Fri 2026-04-03 07:57:26 UTC; 3s ago  
TriggeredBy: ● nginx-logs.timer  
   Process: 3536 ExecStart=/usr/local/bin/nginx-logs.sh /var/log/nginx sakulinim.speedtest.logs (code=exited, status=0/SUCCESS)  
   Main PID: 3536 (code=exited, status=0/SUCCESS)  
    CPU: 3.329s  
  
Apr 03 07:57:23 sakulinim systemd[1]: Starting nginx-logs.service - Upload nginx logs to S3...  
Apr 03 07:57:24 sakulinim nginx-logs.sh[3541]: [161B blob data]  
Apr 03 07:57:25 sakulinim nginx-logs.sh[3562]: [176B blob data]  
Apr 03 07:57:25 sakulinim nginx-logs.sh[3588]: [159B blob data]  
Apr 03 07:57:26 sakulinim nginx-logs.sh[3610]: [174B blob data]  
Apr 03 07:57:26 sakulinim systemd[1]: nginx-logs.service: Deactivated successfully.  
Apr 03 07:57:26 sakulinim systemd[1]: Finished nginx-logs.service - Upload nginx logs to S3.  
Apr 03 07:57:26 sakulinim systemd[1]: nginx-logs.service: Consumed 3.329s CPU time.  
ivan@sakulinim:~$ sudo systemctl start nginx-logs.service  
  
ivan@sakulinim:~$ sudo systemctl list-timers --all | grep nginx  
Fri 2026-04-03 22:00:00 UTC          14h - - nginx-logs.timer  
      nginx-logs.service
```

Рисунок 22 — Статус таймера отправки логов

1.5.7 Общий юнит приложения

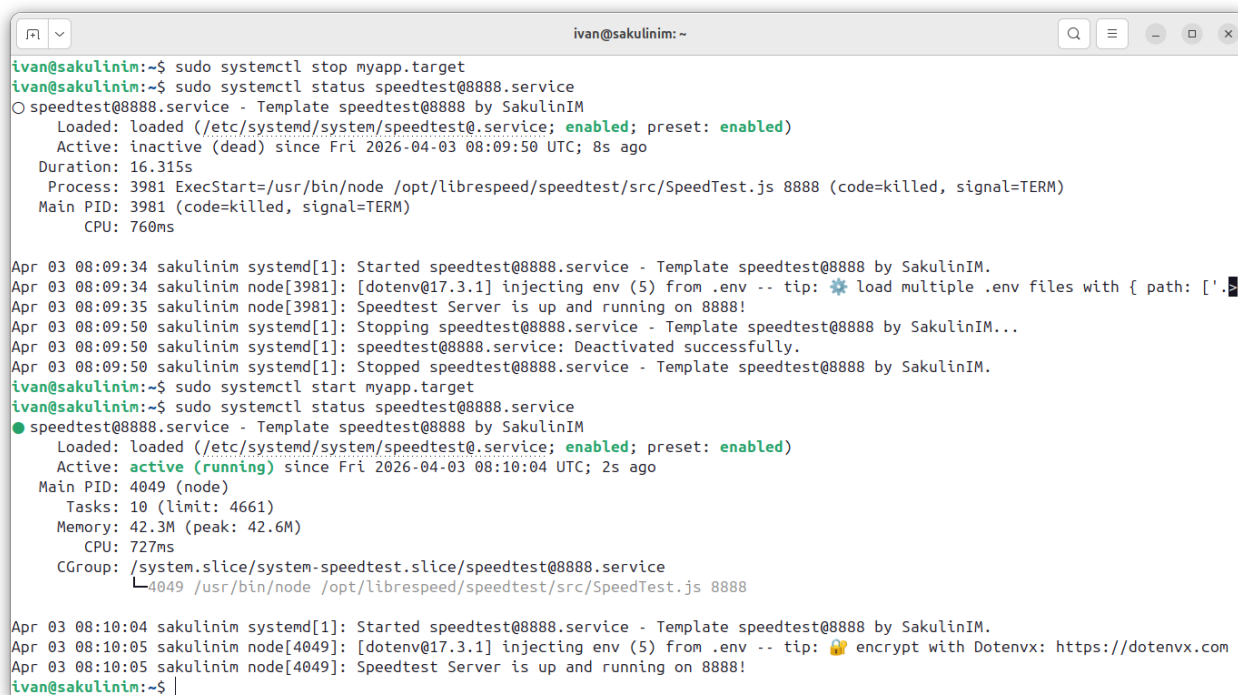
Для управления всеми компонентами приложения (PostgreSQL, nginx-custom, librespeed.target) был создан целевой юнит `myapp.target`. В каждый из сервисов добавлена директива `PartOf=myapp.target`, что позволяет запускать и останавливать всё приложение одной командой. На рисунке 23 показан файл цели.



```
ivan@sakulinim: ~  
GNU nano 7.2 /etc/systemd/system/myapp.target  
[Unit]  
Description=My Application Target  
Requires=nginx-custom.service postgresql.service librespeed.target  
After=nginx-custom.service postgresql.service librespeed.target  
  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 23 — Файл `myapp.target`

Проверка статусов (рисунок 24) подтвердила корректную работу.



```
ivan@sakulinim:~$ sudo systemctl stop myapp.target  
ivan@sakulinim:~$ sudo systemctl status speedtest@8888.service  
● speedtest@8888.service - Template speedtest@8888 by SakulinIM  
   Loaded: loaded (/etc/systemd/system/speedtest@.service; enabled; preset: enabled)  
   Active: inactive (dead) since Fri 2026-04-03 08:09:50 UTC; 8s ago  
 Duration: 16.315s  
    Process: 3981 ExecStart=/usr/bin/node /opt/librespeed/speedtest/src/SpeedTest.js 8888 (code=killed, signal=TERM)  
   Main PID: 3981 (code=killed, signal=TERM)  
      CPU: 760ms  
  
Apr 03 08:09:34 sakulinim systemd[1]: Started speedtest@8888.service - Template speedtest@8888 by SakulinIM.  
Apr 03 08:09:34 sakulinim node[3981]: [dotenv@17.3.1] injecting env (5) from .env -- tip: ⚡ load multiple .env files with { path: ['.']  
Apr 03 08:09:35 sakulinim node[3981]: Speedtest Server is up and running on 8888!  
Apr 03 08:09:50 sakulinim systemd[1]: Stopping speedtest@8888.service - Template speedtest@8888 by SakulinIM..  
Apr 03 08:09:50 sakulinim systemd[1]: speedtest@8888.service: Deactivated successfully.  
Apr 03 08:09:50 sakulinim systemd[1]: Stopped speedtest@8888.service - Template speedtest@8888 by SakulinIM.  
ivan@sakulinim:~$ sudo systemctl start myapp.target  
ivan@sakulinim:~$ sudo systemctl status speedtest@8888.service  
● speedtest@8888.service - Template speedtest@8888 by SakulinIM  
   Loaded: loaded (/etc/systemd/system/speedtest@.service; enabled; preset: enabled)  
   Active: active (running) since Fri 2026-04-03 08:10:04 UTC; 2s ago  
 Main PID: 4049 (node)  
    Tasks: 10 (limit: 4661)  
  Memory: 42.3M (peak: 42.6M)  
     CPU: 727ms  
    CGroup: /system.slice/system-speedtest.slice/speedtest@8888.service  
            └─4049 /usr/bin/node /opt/librespeed/speedtest/src/SpeedTest.js 8888  
  
Apr 03 08:10:04 sakulinim systemd[1]: Started speedtest@8888.service - Template speedtest@8888 by SakulinIM.  
Apr 03 08:10:05 sakulinim node[4049]: [dotenv@17.3.1] injecting env (5) from .env -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com  
Apr 03 08:10:05 sakulinim node[4049]: Speedtest Server is up and running on 8888!  
ivan@sakulinim:~$
```

Рисунок 24 — Проверка статуса `myapp.target`

2 ВОПРОСЫ

1) Что такое юнит (unit) в systemd?

Юнит — это объект, которым управляет systemd. Каждый юнит описывается конфигурационным файлом и может представлять сервис, сокет, таймер, точку монтирования, цель (target) и т.д.

2) В чём разница между командами `systemctl enable` и `systemctl start`?

`enable` создаёт символические ссылки, чтобы юнит автоматически запускался при загрузке системы. `start` запускает юнит немедленно, в текущей сессии.

3) Как просмотреть логи конкретного сервиса?

Используется команда `journalctl -u имя_сервиса.service`. Также можно добавить флаги `-f` для слежения или `-since "today"` для фильтрации по времени.

4) Что такое таймеры (timers) в systemd и чем они отличаются от cron?

Таймеры — это юниты systemd, которые запускают сервисы по расписанию. В отличие от cron, они интегрированы с systemd (логирование, зависимости, изоляция), поддерживают монотонные таймеры (относительно загрузки) и могут обрабатывать пропущенные запуски.

5) Как ограничить потребление памяти для сервиса?

Необходимо создать слайс (slice) с директивой `MemoryMax=` и указать этот слайс в сервисе через `Slice=имя.slice`. Например: `MemoryMax=512M`.

6) Что даёт использование целевых юнитов (target)?

Цели позволяют группировать несколько сервисов и управлять ими как единым целым (запуск, остановка, проверка статуса). Также они используются для определения уровней загрузки (аналог runlevels в SysV init).

7) Как проверить синтаксис юнит-файла без его применения?

Команда `systemd-analyze verify /путь/к/файлу.service` выполняет статическую проверку конфигурации.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы был получен практический опыт работы с systemd: анализ времени загрузки и зависимостей, управление юнитами, создание пользовательских сервисов и таймеров, настройка ограничений ресурсов (слайсы), группировка сервисов с помощью целевых юнитов. Были реализованы автоматические задачи резервного копирования PostgreSQL и отправки логов Nginx в S3 с использованием таймеров. Все компоненты приложения объединены в общую цель `myapp.target`, что упрощает управление. Полученные навыки необходимы для администрирования современных Linux-систем, использующих systemd.